

Deployment & DevOps Module

Full-Stack Development

Git Workflows

Git tracks changes to code over time and lets multiple people work on the same codebase without overwriting each other's work. The core workflow most teams use: create a feature branch off the main branch for each piece of work, commit changes with clear messages describing what and why, push the branch and open a pull request for review, then merge back into main once approved. Branching this way keeps main always in a working, deployable state, and gives every change a reviewable, revertible history — essential once more than one person touches the code.

Environment Configuration

Real applications run in multiple environments — local development, staging (a production-like environment for final testing), and production (what real users actually use) — and each needs different configuration: different database connection strings, API keys, and feature flags. The standard practice is to never hard-code these values in source code; instead, read them from environment variables at runtime, with a separate `.env` file (excluded from version control) per environment. This means the exact same code can run correctly in every environment just by changing its configuration, not its logic.

Containerization Basics

A container packages an application together with everything it needs to run — code, runtime, system libraries, dependencies — into one portable unit that behaves identically wherever it runs. Docker is the standard tool for this: a `Dockerfile` describes how to build an image (a snapshot of the app plus its environment), and that image can then be run as a container on any machine with Docker installed, eliminating the classic 'it works on my machine' problem caused by environment differences between developers' laptops and servers.

CI/CD Fundamentals

Continuous Integration (CI) automatically builds and tests every code change (usually on every pull request) so problems are caught immediately rather than after merging. Continuous Deployment/Delivery (CD) automatically ships code that passes those checks to staging or production, removing manual, error-prone deployment steps. A typical CI/CD pipeline runs: install dependencies, run the linter and automated tests, build the application, and — if every step passes — deploy it. This automation is what lets teams ship changes frequently and confidently instead of treating every deployment as a risky, manual event.

Cloud Deployment

Deploying an application means making it reachable to real users outside your local machine. At a minimum this involves: hosting the built application on a server (a traditional VM, a managed platform like a PaaS, or a containerized environment), configuring a domain name and HTTPS certificate so users can reach it securely, and connecting it to a production database (usually separate from any development database, with its own credentials and backups). Modern deployment also means planning for what happens when something goes wrong — monitoring/logging to know when the app is broken, and a way to quickly roll back to the previous working version if a deployment introduces a bug.

Learn More

Git Official Documentation

<https://git-scm.com/doc>

Docker Official Documentation

<https://docs.docker.com/>

GitHub Actions Documentation

<https://docs.github.com/en/actions>